PrepPressure

Weekly Mini Project 03 — Team 3 · Project Specification

Project	PrepPressure
Team leader	Laith Haj Hosin
Team members	Shiam Ezzo, Esmaeel Mahfoud, Badr Al Jerf, Adham Albasha, Hassan Alloush

> Practice technical interviews against the clock — and get a score you can actually explain.

1. Project Overview

Many juniors know the material but freeze in a real technical interview: the clock is running, the questions are unfamiliar, and there is no second attempt. The best preparation is practising under similar pressure.

PrepPressure is a simulated technical interview engine. A candidate chooses an interview type and difficulty, receives a timed set of questions drawn from a prepared question bank, answers them one by one, and receives a calculated, explainable result.

This is **not** an AI interviewer. There is no external AI service and no code execution. Questions come from your team's own question bank, and scoring follows fixed, deterministic rules.

2. Team

The team leader and members are listed in the table at the top of this document.

Team Responsibility

- The leader coordinates the internal work split and makes sure the team reaches a working integrated result.
- Frontend/backend ownership is not assigned by the instructor.
- The team should decide how responsibilities are distributed.
- Every member must make a meaningful contribution.
- Every member should understand the project at a high level.
- Every member must participate in the final English presentation.

3. Project Goal

Build a web application where a candidate can:

- browse the question bank by category, difficulty and type (without seeing answers);
- start a timed interview session for a chosen category and difficulty;
- answer multiple-choice and short-answer questions one at a time, with a visible timer;
- finish the interview, or have it end automatically when time runs out;
- receive a scored, explained result and review past interview sessions.

The backend stores every interview session with the timestamps and state needed to enforce its lifecycle, and calculates every score.

4. Technology Constraints

Area	Requirement
Frontend	React + TypeScript
Backend	NestJS
Database	MongoDB, through Mongoose
API documentation	Swagger / OpenAPI

- All application data is stored in MongoDB and must survive a backend restart.
- How you organize your React app and your NestJS backend is your team's decision.
- The visible countdown runs in the frontend, but the backend's stored timestamps decide whether an answer arrived in time.
- Do **not** add user accounts, login or passwords, another database, external AI services, code execution, cloud services or paid APIs.

5. User Scenario

Sara has a junior frontend interview next week. She opens PrepPressure, types her display name, chooses **React**, difficulty **Medium**, and **6 questions**. The session starts; she has 9 minutes in total, and a countdown is shown.

Question 1 is multiple choice — she picks an answer and moves on. Question 2 is a short answer: "Why does React need a key when rendering a list?" She writes a few sentences. Accidentally, she tries to submit question 2 again — the application tells her it is already answered.

With 30 seconds left she still has one question open. Time runs out; the session ends automatically and the unanswered question scores zero.

Her result: **8.0 / 12 points (66.7%) — "Almost ready"**. For the short-answer question she sees which expected keywords she used ("identity", "re-render") and which she missed ("reorder"). She opens History and compares this attempt with her previous one.

6. Required Features

Feature	Description
Question bank	Browse active questions, filtered by category, difficulty and type. Answers are never shown here.
Interview setup	Choose display name, category, difficulty and number of questions.
Start session	Create a session with randomly selected questions and a time limit.
Answer questions	Answer one question at a time; see progress and the remaining time.
Finish session	Finish manually, or automatically when the time limit is reached.
Result	Calculated score, percentage, level and per-question explanation.
History	All past sessions, filtered by candidate name, category and status.
Question management	Create, edit and deactivate questions — internal operations.

Question information

Each question must have at least: text, category (for example React, TypeScript, Node.js, NestJS, MongoDB, Git), difficulty (**Easy**, **Medium**, **Hard**), type (**Multiple choice** or **Short answer**), and whether it is active.

- A multiple-choice question has 2–6 answer choices and exactly one correct choice.

- A short-answer question has a list of 2–6 expected keywords and the number of keywords required for full points.

Interview session information

Each session must have at least: candidate display name, category, difficulty, its questions, its answers, status (**In progress**, **Finished** or **Timed out**), start time, deadline and finish time.

7. Business Rules

Starting a session

1. A session has between **5 and 10 questions**.
2. Questions are selected **randomly** from **active** questions that match the chosen category and difficulty. A question never appears twice in the same session.
3. If not enough matching active questions exist, starting is rejected and the user is told how many are available.
4. The questions of a session are **fixed when it starts**. Editing or deactivating a question later must not change sessions that already exist.
5. Time allowed per question: **Easy 60 s**, **Medium 90 s**, **Hard 120 s**. The session deadline is the start time plus the total for all its questions.

Answering

1. Answers are accepted only while the session is **In progress** and before its deadline. A grace period of up to **5 seconds** is allowed for network delay.
2. Each question can be answered **once**. A second answer to the same question is rejected.
3. An answer must belong to a question in this session. A multiple-choice answer must be one of that question's choices. A short answer cannot be empty and is at most 1,000 characters.
4. While a session is in progress, the correct choice and expected keywords are **never** sent to the candidate.

Ending

1. The candidate can finish the session at any time. Its status becomes **Finished**.
2. If the deadline has passed, the session is treated as **Timed out** from that moment: no more answers are accepted, and the next time the session is requested it is shown as Timed out with its result.
3. A Finished or Timed-out session cannot be changed in any way.
4. Results, correct answers and explanations are available only after the session has ended.

Scoring

1. Points per question: **Easy 1**, **Medium 2**, **Hard 3**.
2. **Multiple choice:** full points for the correct choice, otherwise 0.
3. **Short answer:** an expected keyword counts as used if it appears in the answer, ignoring upper/lower case. Full points if the number of used keywords is at least the required number; **half points** if at least one keyword is used but fewer than required; 0 if none.
4. Unanswered questions score 0.
5. The percentage is points earned ÷ maximum points × 100, shown with one decimal place.
6. Level: **Ready** at 85% or more, **Almost ready** from 60% to below 85%, **Needs practice** below 60%.
7. The same answers must always produce exactly the same score.

Question bank

1. Creating, editing and deactivating questions are internal operations (see section 13).

2. A question that already appears in any session cannot be deleted — it can only be deactivated. Deactivated questions are not used in new sessions.

8. Required Screens / Views

Screen	Must show / allow
Home / Setup	Display name, category, difficulty and number of questions; how many matching questions are available; a start action.
Interview	One question at a time, its type and points; progress ("Question 3 of 6"); the countdown; answer input; next and finish actions.
Result	Score, maximum, percentage, level, time used, unanswered count; per-question review with the candidate's answer, the correct answer or keywords used/missed, and the points earned; breakdown by difficulty.
History	Past sessions with name, category, difficulty, status, score, percentage and date; filters; a way to open each result.
Question Bank	Active questions with category, difficulty and type, and the filters — without answers.

All screens must be reachable through the application's navigation.

9. Required User Flows

Flow A — Start an interview

1. The user chooses category React, difficulty Medium and 6 questions.
2. The setup screen shows how many matching questions exist.
3. The session starts; the first question and the countdown appear.
4. Asking for 10 questions when only 7 exist is rejected with a clear message.

Flow B — Answer under pressure

1. The user answers a multiple-choice question and a short-answer question.
2. Re-submitting an answer to the same question is rejected.
3. The progress indicator updates after every answer.

Flow C — Finish and understand the result

1. The user finishes the session.
2. The result shows the total, percentage and level.
3. For each short-answer question the user sees which keywords were used and missed, and why it earned full, half or zero points.

Flow D — Run out of time

1. The user lets the countdown reach zero.
2. A late answer is rejected.
3. The session is shown as Timed out and its result counts unanswered questions as 0.

Flow E — Review history

1. The user opens History and filters by their name.
2. The user opens an older session and sees its stored result.

10. Required Calculations / Derived Information

Derived information	Where it appears
Number of matching active questions	Home / Setup
Session deadline and remaining time	Interview
Progress (answered / total)	Interview
Points per question and keyword matches	Result
Total, maximum, percentage and level	Result, History
Breakdown by difficulty (points earned / available)	Result
Time used and number of unanswered questions	Result
Session status, including automatic Timed out	Interview, Result, History

11. Validation & Error Scenarios

General expectations

- Required information cannot be missing; numbers must be sensible.
- Invalid actions must be rejected — the system must never silently accept an invalid operation.
- The user always receives a clear, human-readable message explaining **why** an action failed.
- Something that does not exist is reported as "not found", not as an empty success.
- A request that conflicts with the current state (for example answering twice) is reported as a conflict.
- When a request cannot be completed, the screen shows a sensible message — never a blank page.

Project-specific scenarios

Situation	Expected behavior
Display name missing or only spaces	Rejected.
Unknown category, difficulty or type	Rejected.
Number of questions outside 5–10	Rejected.
Not enough matching active questions	Rejected; the number available is reported.
Answering a question twice	Rejected as a conflict.
Answering a question that is not in this session	Rejected.
Multiple-choice answer that is not one of the choices	Rejected.
Empty short answer, or longer than 1,000 characters	Rejected.
Answering after the deadline (plus grace period)	Rejected; the session is Timed out.
Answering or finishing an ended session	Rejected as a conflict.
Requesting the result of a session still in progress	Rejected.
A session or question that does not exist	"Not found".
A multiple-choice question without exactly one correct choice	Rejected.
Deleting a question used in a session	Rejected as a conflict; it can be deactivated instead.

12. Data & Persistence Requirements

- Questions and interview sessions are stored in MongoDB.

- On first start with an empty database, the backend seeds at least **40 questions** written by your team, covering at least **4 categories**, all **3 difficulties** and **both types**, so any demo combination has enough questions.
- Records are identified and linked using the **numeric, application-level id convention used in class** (as in B4F Board).

Data Design Decisions

Your team must make, implement and be able to explain these decisions:

1. **Relationship decision.** A session must not change when the question bank changes (rule 4). Decide how a session stores its questions and answers — copied into the session, or linked to the question bank by id — and how you guarantee that rule. Write down your choice and why.
2. **Index decision.** Identify a query that runs often — for example selecting active questions by category and difficulty, or listing History by candidate name, newest first — and add an index that supports it. Be ready to explain which fields it covers and why.

Both decisions must be written in your repository's README.

13. Backend Quality Requirements

- **Validation:** every request body and every filter query parameter is validated with DTOs and the validation approach taught in class. Unknown fields are rejected.
- **Meaningful errors:** failures use the correct kind of HTTP error (bad request, not found, conflict) with a clear message.
- **Logger:** log the events that matter — sessions started (category, difficulty, number of questions), sessions finished or timed out (with the score), late answers rejected, and question-bank changes. Do **not** log every request.
- **Swagger:** every endpoint is documented with a short summary, grouped by resource, with its request body and query parameters visible.
- **Internal-only actions:** creating, editing, deactivating and deleting **questions** must be protected with the `InternalToolGuard` approach taught in class (documented in Swagger, including the required header). Browsing the question bank and taking interviews must **not** be guarded.
- **No fake logic:** scores, levels and timeouts must come from real calculations on stored data and stored timestamps. Hard-coded results do not count.

14. Out of Scope

The following are **not required** and must not be added to the core project: AI grading or any external AI service; running or judging code; user accounts, login, passwords or JWT; Mongoose ObjectId references or `populate`; aggregation pipelines; pagination; WebSockets; another database; deployment.

15. Minimum Deliverable (CORE)

Your project counts as complete only when **all** of the following work:

- □ A working React + TypeScript frontend and a working NestJS backend, integrated with each other.
- □ Data is stored in MongoDB and survives a backend restart; the question bank is seeded.
- □ Every required screen in section 8 exists and is reachable through the navigation.
- □ Every business rule in section 7 is enforced.
- □ Sessions start with random, non-repeating, matching questions, or are rejected when too few exist.
- □ Answers are accepted once, only in time, and never reveal correct answers early.
- □ Finish and automatic timeout both end the session correctly.

- □ Scoring, percentage, level and per-question explanations are correct.
- □ Editing the question bank does not change existing sessions.
- □ Questions are managed only through guarded internal endpoints.
- □ Validation, error messages, Logger and Swagger meet section 13.
- □ Both data design decisions are implemented and written in the README.
- □ The README explains how to install, configure (environment variables) and run both parts.

16. Stretch Features (optional)

Only start these after every CORE item works. They never replace a missing CORE requirement.

- **Mixed interview** — choose several categories in one session.
- **Personal best** — show each candidate's best percentage per category in History.
- **Weak-spot summary** — after a session, list the categories or difficulties where the candidate lost the most points.
- **Question management screen** — an internal screen that sends the internal-tool header.

17. Presentation & Demo Acceptance

Your team presents **in English**; every member speaks. Your instructor will confirm the date and time limit. Present the problem, a short product walkthrough, a live demo, your data design decisions and the most important business rules.

The live demo must pass these acceptance tests:

1. Requesting more questions than are available is rejected with the available count.
2. While a session is in progress, the data the candidate receives contains no correct answers or keywords.
3. Answering the same question twice is rejected.
4. One short answer earns half points, and the result explains which keywords were used and missed.
5. A session that runs out of time rejects a late answer and appears as Timed out.
6. A result can be checked by hand against the scoring rules.
7. Editing a question used in an old session does not change that session's review.
8. In Swagger, creating a question without the internal-tool header is rejected, and with it succeeds.
9. Restarting the backend keeps all data.

18. Evaluation Focus

Area	What we look for
Functional completeness	The required features and screens exist and work.
Correct business behavior	Session lifecycle, timing and scoring follow the rules exactly.
Frontend/backend integration	The two parts genuinely work together on real stored data.
Backend quality	Validation, meaningful errors, Logger, Swagger and correct guard placement.
Data design	Sensible, explained relationship and index decisions.
User experience	A clear, calm interview screen and an understandable result.
Team collaboration and Git	Shared work, visible contributions from every member.
Explaining your own work	Every member can explain the product, its rules and its design.
Presentation quality	Clear English presentation with a working live demo.